

Program 2:

Develop a multi-stage Dockerfile for container orchestration

31/10/2025

Project Directory:

```
prog2
|- node_modules
|- src
|  |- index.js
|  |- Dockerfile
|  |- package.json
|  |- package-lock.json
```

Program Code and steps for execution:

Step 1: Move to the directory where we work for DevOps lab. Then create a project directory with the name prog2

```
> mkdir prog2
```

```
> cd prog2/
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~$ cd devops_lab/
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab$ mkdir prog2
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab$ cd prog2/
```

Step 2: Initialize the npm, also install the express by using the command,

```
> npm init -y
```

```
> npm install express
```

This add node_modules, package.json and package-lock.json to the prog2 directory

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ npm init -y
Wrote to /home/chinnu/devops_lab/prog2/package.json:
```

```
{
  "name": "prog2",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ npm install express
```

```
added 65 packages, and audited 66 packages in 6s
```

```
20 packages are looking for funding
  run `npm fund` for details
```

```
found 0 vulnerabilities
```

Step 3: Edit the package.json in such a way that it should contain the below code exactly

```

1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ nano package.json
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ cat package.json
{
  "name": "prog2",
  "version": "1.0.0",
  "description": "Simple Express app for Docker multi-stage build",
  "main": "src/index.js",
  "scripts": {
    "start": "node src/index.js",
    "dev": "NODE_ENV=development node src/index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^5.2.1"
  }
}

```

Step 4: Create a folder/directory with the name src inside the prog2 folder, then write the index.js file inside the src/

The index.js content is shown in below image

```

1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ mkdir src
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ cd src/
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2/src$ sudo nano index.js
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2/src$ cat index.js
const express = require('express');
const app = express();

const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Hello from prog2 running in Docker!');
});

app.listen(PORT, () => {
  console.log(`Server listening on port ${PORT}`);
});

```

Step 5: Come out of the src/ folder and create the Dockerfile inside the prog2/

The Dockerfile contains two stages and the content is in below image

```

1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2/src$ cd ..
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ sudo nano Dockerfile
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ cat Dockerfile
# Stage 1: Build Stage
FROM node:18-alpine AS builder

# Set working directory inside the image
WORKDIR /app

# Copy package definition files and install dependencies
COPY package.json package-lock.json ./
RUN npm ci --only=production

# Copy source code
COPY src ./src

# Stage 2: Production Stage
FROM node:18-alpine AS runner
WORKDIR /app

# Set env for production
ENV NODE_ENV=production
ENV PORT=3000

# Copy only what we need from builder
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/src ./src
COPY package.json ./

# Expose the port
EXPOSE 3000

# Default command to start the app
CMD ["node", "src/index.js"]

```

Step 6: The final Project directory looks like the below

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ ls
Dockerfile  node_modules  package.json  package-lock.json  src
```

Step 7: Build and Run the image using the below commands,

> docker build -t prog2 .

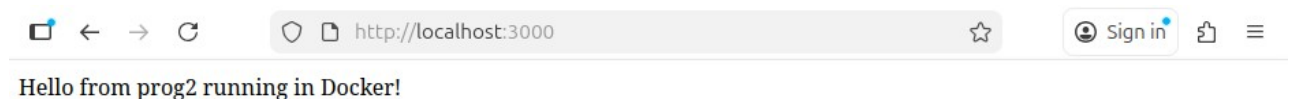
> docker run -it -p 3000:3000 prog2

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ docker build -t prog2 .
[+] Building 8.3s (14/14) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 689B                             0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine 4.7s
=> [auth] library/node:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                 0.0s
=> => transferring context: 2B                                    0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 27.93kB                              0.0s
=> [builder 1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663 0.0s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3 0.0s
=> CACHED [builder 2/5] WORKDIR /app                             0.0s
=> [builder 3/5] COPY package.json package-lock.json ./         0.0s
=> [builder 4/5] RUN npm ci --only=production                   2.4s
=> [builder 5/5] COPY src ./src                                  0.1s
=> [runner 3/5] COPY --from=builder /app/node_modules ./node_modules 0.1s
=> [runner 4/5] COPY --from=builder /app/src ./src              0.1s
=> [runner 5/5] COPY package.json ./                             0.0s
=> exporting to image                                           0.6s
=> => exporting layers                                           0.3s
=> => exporting manifest sha256:3d4326a32bf466dc13daa55c9a19cf0fa5ac8b8f 0.0s
=> => exporting config sha256:9747d63511b81cf0f85525b4cb41b723f5199f1982 0.0s
=> => exporting attestation manifest sha256:2d231c91a6923155ae94b890867e 0.0s
=> => exporting manifest list sha256:5ffbcd019fed3012c52a36d16412ca24228 0.0s
=> => naming to docker.io/library/prog2:latest                  0.0s
=> => unpacking to docker.io/library/prog2:latest               0.2s
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ docker run -it -p 3000:3000 prog2
Server listening on port 3000
```

Step 8: Open any browser in the local machine and search for,

> <http://localhost:3000/>

The output will be as show below



Step 9: Stop and remove all the running container, also remove the created image by using the command,

> docker images

> docker ps

> docker container stop <4_characters_of_container_id>

> docker container rm <4_characters_of_container_id>

> docker rmi <4_characters_of_image_id>

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ docker images
IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
prog2:latest   5ffbcd019fed      186MB       45.6MB        U
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
bb0ae779f0a2   prog2         "docker-entrypoint.s..." 3 minutes ago  Up 3 minutes  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  nervous_bartik
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ docker container stop bb0ae
bb0ae
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ docker container rm bb0ae
bb0ae
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog2$ docker rmi prog2:latest
Untagged: prog2:latest
Deleted: sha256:5ffbcd019fed3012c52a36d16412ca24228e8f0d26852979fc557580037d770f
```